



Orviohub Team & Branch Management: Full Implementation Plan

Executive Summary

Orviohub will implement a **hybrid team management model** combining organization-level membership (foundation) with application-branch-level role assignments (operational). This approach mirrors GitHub's organization structure while supporting your multi-app, multi-branch architecture. Users are invited to organizations first, then assigned to specific application-branches with granular roles and permissions. ^{[1] [2] [3] [4]}

1. System Architecture Overview

1.1 Core Principles

- **Organization-first membership:** Users must be org members before accessing any app-branch ^{[2] [1]}
- **Application isolation:** Staff in Inventory cannot be moved to Task Management without explicit cross-app assignment ^[5]
- **Branch-level operational flexibility:** Branch owners can transfer staff between branches within the same app ^[1]
- **GitHub-style invite UX:** Search by email → view profile → send invite with role/branch selection ^{[4] [6]}
- **Dual notifications:** In-app + email for all invitations and transfers ^{[6] [1]}

1.2 Data Model Relationships

```
User (personal account)
  ↓
orgMembers (org-level membership: owner/admin/member)
  ↓
orgApplicationMembers (app-branch assignments: cashier/manager/staff)
  ↓
branches (physical locations per org-app)
```

Key constraint: A user can only be assigned to an app-branch if they already exist in `orgMembers` for that organization. ^{[2] [1]}

2. User Journeys & Stories

2.1 Primary User Personas

Persona	Description	Goals
Branch Owner (Ola)	Owns "Code X Stores" org, manages Inventory app across 3 branches	Invite staff, assign roles, transfer between branches, monitor access
Existing User (Mary)	Has Orviahub account, works at 2 orgs, cashier at Branch A	Accept invites, switch branches, view assignments
New Invitee (John)	No Orviahub account yet, invited as cashier at Branch B	Create account, accept invite, start work
Transferred Staff (Peter)	Cashier at Branch A, moving to Branch B as manager	Understand transfer, accept new role, maintain access

2.2 Journey 1: Branch Owner Invites New Staff (GitHub-Style Flow)

Actor: Ola (Branch Owner, Code X Stores, Inventory app)

Precondition: Ola is logged into `inventory.orviahub.localhost`, active workspace = Code X Stores, Inventory app activated, Branch A selected

Step-by-Step Flow

Step 1: Navigate to Team Management

- Ola clicks "Team" in Inventory sidebar
- Frontend calls GET
`/v1/workspaces/:workspaceId/products/inventory/branches/:branchId/members`
- Displays current branch members list^[1]

Step 2: Initiate Invite

- Ola clicks "Invite Member" button
- Modal opens: "Invite to Branch A - Inventory"
- Form fields:
 - Email address (required, with autocomplete from previous invites)
 - Role dropdown (Cashier, Manager, Stock Keeper, Viewer)^[7]
 - Branch selector (defaults to current branch, but can select other branches in same org)
 - Optional message (e.g., "Welcome to the team! Your shift starts Monday.")
 - Invitation expiry (default: 7 days, matches GitHub standard)^[4] ^[6]

Step 3: Search & Validate

- Ola types "john.doe@gmail.com"
- Frontend debounced call: GET `/v1/users/search?email=john.doe@gmail.com`

- Backend checks users table by email (normalized, case-insensitive)
- **Scenario A: User exists**
 - Returns: { `userId: "usr_456", name: "John Doe", avatarUrl: "...", hasAccount: true` }
 - Modal shows: "✓ John Doe (john.doe@gmail.com) - Orviohub member"
 - Ola sees profile preview (name, avatar, current orgs if public)
- **Scenario B: User doesn't exist**
 - Returns: { `hasAccount: false` }
 - Modal shows: "✗ john.doe@gmail.com - Will create account on accept"
 - Warning: "Invitee will need to create an Orviohub account"

Step 4: Backend Pre-Checks (before sending)

On "Send Invite" click, frontend calls POST `/v1/workspaces/:workspaceId/invitations` with:

```
{
  "email": "john.doe@gmail.com",
  "role": "cashier",
  "branchId": "br_789",
  "applicationId": "app_inventory",
  "message": "Welcome to the team!",
  "expiresInDays": 7
}
```

Backend validation:

1. ✓ Ola has `inventory.manage_members` permission in this org^[8]
2. ✓ Email is valid format, normalized to lowercase
3. ✓ No active pending invitation for this email + branch combo
4. ✓ User (if exists) is already in `orgMembers` for this org
 - If NOT in `orgMembers`: Auto-create pending org membership invitation first^[3] ^[2]
5. ✓ Free Trial org limits: ≤1 branch per app (if applicable)^[9]
6. ✓ Invitee not already an active member in this branch

Step 5: Create Invitation

Backend transaction:

1. Generate cryptographically secure token (e.g., `inv_token_abc123xyz`)
2. Hash token (SHA-256) → store `tokenHash` in `workspaceInvitations` table^[3]
3. Insert invitation row:

```
workspaceId: "ws_123"
applicationId: "app_inventory"
branchId: "br_789"
email: "john.doe@gmail.com"
emailNormalized: "john.doe@gmail.com"
```

```
inviteeUserId: "usr_456" (if exists, else null)
role: "cashier"
tokenHash: "sha256_hash_here"
status: "pending"
invitedBy: "usr_100" (Ola's userId)
expiresAt: now + 7 days
createdAt: now
```

4. Write audit log: `workspace.member_invited`

5. Return: { `invitationId: "inv_789"`, `token: "inv_token_abc123xyz"` }

Step 6: Send Notifications

Backend triggers async jobs:

Email notification (via SendGrid/Resend):

- **Subject:** "Ola invited you to join Code X Stores as Cashier (Inventory - Branch A)"
- **Preview text:** "Review your invitation and accept to start managing inventory"
- **Body:**
 - Workspace logo + name: "Code X Stores"
 - Inviter: "Ola (Branch Owner)"
 - Role: "Cashier"
 - Application: "Inventory"
 - Branch: "Branch A"
 - Message: "Welcome to the team!"
 - Expiry: "This invitation expires on Sep 16, 2026"
 - **Primary CTA button:** "Review Invitation" → https://accounts.orviohub.localhost/invitations/inv_token_abc123xyz
 - Secondary link: "Decline" (if already has account)
 - Footer: "You received this because Ola invited you. Orviohub, Abuja, NG"

In-app notification (if user exists):

- Insert into notifications table :

```
userId: "usr_456"
type: "invitation.received"
title: "New invitation from Code X Stores"
body: "Ola invited you as Cashier in Inventory (Branch A)"
link: "/invitations/inv_token_abc123xyz"
read: false
createdAt: now
```

- Real-time push via Convex subscription or WebSocket

Step 7: Confirmation UI

- Modal closes, shows toast: "✓ Invitation sent to john.doe@gmail.com"

- Team list refreshes, shows pending invitation badge: "John Doe (Pending)"^[10]
- Ola can now: Resend, Revoke, or View details

2.3 Journey 2: Invitee Accepts Invitation

Actor: John (invitee, may or may not have Orviohub account)

Precondition: John received email with invitation link

Scenario A: John Already Has Orviohub Account

Step 1: Click Invitation Link

- John clicks "Review Invitation" in email
- Redirects to: `https://accounts.orviohub.localhost/invitations/inv_token_abc123xyz`
- Frontend calls `GET /v1/invitations/:token` (token sent in path, backend hashes it)

Step 2: Backend Validation

Backend checks:

1. ✓ Token hash exists in `workspaceInvitations` table
2. ✓ Status = "pending"
3. ✓ `expiresAt` > now (not expired)
4. ✓ `revokedAt` is null (not revoked by sender)
5. ✓ Requesting user is authenticated (session cookie valid)
6. ✓ Authenticated user's email matches invitation email (or `userId` matches `inviteeUserId`)^[3]

Step 3: Display Invitation Screen

UI shows:

- Workspace logo + name: "Code X Stores"
- Inviter: "Ola (Branch Owner)"
- Application: "Inventory"
- Branch: "Branch A"
- Role: "Cashier"
- Permissions preview: "Can sell products, view stock, process orders"^[7]
- Expiry countdown: "Expires in 6 days"
- Two buttons:
 - ✓ "Accept Invitation" (primary, green)
 - ✕ "Decline" (secondary, gray)

Step 4: Accept Invitation

John clicks "Accept" → `POST /v1/invitations/:token/accept`

Backend transaction:

1. Re-validate all checks from Step 2
2. Check if John exists in orgMembers for this org:
 - If NO: Auto-create org membership first:

```
orgMembers.insert({
  organizationId: "ws_123",
  userId: "usr_456",
  role: "member",
  status: "active",
  invitedBy: "usr_100",
  acceptedAt: now
})
```

- If YES: Proceed (already org member)

3. Create app-branch membership:

```
orgApplicationMembers.insert({
  organizationId: "ws_123",
  applicationId: "app_inventory",
  branchId: "br_789",
  userId: "usr_456",
  role: "cashier",
  status: "active",
  assignedBy: "usr_100",
  assignedAt: now
})
```

4. Update invitation status: status = "accepted", acceptedAt = now

5. Revoke the token (single-use)^[4] ^[3]

6. Write audit logs:

- workspace.invitation_accepted
- workspace.member_joined (if new org member)
- branch.member_added

7. Trigger notifications:

- To Ola (inviter): "John accepted your invitation and joined as Cashier"
- To John: "Welcome to Code X Stores! You can now access Inventory (Branch A)"
- Email + in-app

8. Return: { success: true, redirectUrl: "/dashboard" }

Step 5: Redirect to Dashboard

- Frontend redirects John to inventory.orvionhub.localhost/dashboard
- Backend resolves workspace context:

```
{
  "workspace": { "id": "ws_123", "name": "Code X Stores" },
  "membership": { "role": "member", "status": "active" },
  "products": [{ "key": "inventory", "status": "active" }],
  "branchAssignments": [{ "branchId": "br_789", "branchName": "Branch A", "role": "member", "permissions": ["inventory.sell", "inventory.view_stock"] }],
}
```

- John lands on Inventory dashboard, Branch A context active

Scenario B: John Does NOT Have Orvionhub Account

Step 1-3: Same as Scenario A (click link, validate, display invitation)

Step 4: Create Account Flow

- Invitation screen shows: "You need an Orvionhub account to accept this invitation"
- **Form:**
 - Full name (pre-filled if email was shared)
 - Email (pre-filled from invitation, read-only)
 - Password (min 8 chars, strength meter)
 - Checkbox: "I agree to Terms & Privacy Policy"
 - **Button:** "Create Account & Accept Invitation"

Step 5: Account Creation + Acceptance (Atomic)

POST /v1/invitations/:token/accept-with-signup

Backend transaction:

1. Validate invitation token (same as Step 2 in Scenario A)
2. Validate password strength
3. Check email not already registered
4. Create user account:

```
users.insert({
  email: "john.doe@gmail.com",
  emailNormalized: "john.doe@gmail.com",
  name: "John Doe",
  emailVerified: false, // requires verification
  createdAt: now
})
```

5. Send email verification (separate flow, but allow invitation acceptance before verification for UX)
6. Proceed with Steps 2-8 from Scenario A (create org membership + app-branch membership)

7. Auto-login: Set session cookie (wildcard *.orvionhub.localhost) ^[11]

8. Return: { success: true, redirectUrl: "/dashboard" }

Step 6: Onboarding Redirect

- John lands on personal onboarding (if first-time user) ^[12] ^[13]
- After onboarding, redirects to Inventory dashboard (Branch A)

2.4 Journey 3: Transfer Staff Between Branches (Same App)

Actor: Ola (Branch Owner)

Precondition: Peter is active cashier at Branch A, Ola wants to move him to Branch B as manager

Step-by-Step Flow

Step 1: Initiate Transfer

- Ola navigates to "Team" → Branch A members list
- Finds Peter's row, clicks "⋮" (actions menu)
- Selects "Transfer to another branch"
- Modal opens: "Transfer Peter Johnson"

Step 2: Configure Transfer

Modal form:

- **From:** Branch A (read-only)
- **To:** Branch selector dropdown (shows all branches in this org + Inventory app)
 - Options: Branch A (current), Branch B, Branch C, Warehouse
- **New role:** Dropdown (defaults to current role "Cashier", but can change)
 - Options: Cashier, Manager, Stock Keeper, Viewer
- **Effective date:** Date picker (defaults to today, can schedule future transfer)
- **Message to Peter:** "You're being transferred to Branch B as Manager. Congratulations!"
- **Checkbox:** "Notify Peter immediately" (checked by default)

Step 3: Backend Validation

POST /v1/workspaces/:workspaceId/products/inventory/members/:membershipId/transfer

Payload:

```
{
  "fromBranchId": "br_789",
  "toBranchId": "br_790",
  "newRole": "manager",
  "effectiveDate": "2026-09-10",
  "message": "You're being transferred..."
```

```
"notifyImmediately": true
}
```

Backend checks:

1. ✓ Ola has `inventory.manage_members` permission
2. ✓ Peter exists in `orgApplicationMembers` for (org, Inventory, Branch A)
3. ✓ Target branch (Branch B) exists in same org + app
4. ✓ Peter not already active in Branch B (prevent duplicates)
5. ✓ Free Trial limits: org still ≤ 1 branch per app (transfer doesn't change count)
6. ✓ New role is valid for Inventory app^[7]

Step 4: Execute Transfer

Backend transaction:

1. Update `orgApplicationMembers` row:

```
branchId: "br_790" (was "br_789")
role: "manager" (was "cashier")
updatedAt: now
```

2. Insert audit trail row (optional, for history):

```
orgApplicationMemberTransfers.insert({
  memberId: "mem_456",
  userId: "usr_456",
  fromBranchId: "br_789",
  toBranchId: "br_790",
  fromRole: "cashier",
  toRole: "manager",
  transferredBy: "usr_100",
  effectiveDate: "2026-09-10",
  createdAt: now
})
```

3. Write audit logs:

- `branch.member_transferred` (severity: medium)
- `branch.role_changed` (if role changed)

4. If `notifyImmediately = true`:

- **In-app notification to Peter:**

```
type: "branch.transferred"
title: "You've been transferred to Branch B"
body: "Ola transferred you from Branch A to Branch B as Manager. Effective: S
link: "/inventory/branches/br_790"
```

- **Email to Peter:**

- Subject: "You've been transferred to Branch B - Code X Stores"

- Body: Details of transfer, new role, effective date, CTA: "View Branch B Dashboard"
- Notification to Ola: "✓ Peter transferred to Branch B as Manager"

Step 5: UI Updates

- Branch A members list refreshes: Peter no longer appears
- Branch B members list refreshes: Peter appears as "Manager (Transferred today)"
- Toast: "✓ Peter Johnson transferred to Branch B as Manager"

Step 6: Peter's Experience

- Peter receives notification (in-app + email)
- Clicks notification → redirects to Inventory dashboard
- Workspace context now shows Branch B as active branch
- Peter can manually switch back to Branch A if he has multi-branch access (optional feature)

2.5 Journey 4: Cross-App Movement (Inventory → Task Management)

Actor: Ola (Branch Owner) + Admin approval required

Precondition: Peter is cashier in Inventory (Branch A), Ola wants to add him to Task Management app

Key Constraint

Staff in Inventory cannot be automatically moved to Task Management. This requires:

- Explicit cross-app assignment (separate from branch transfer)
- Org-admin or owner approval (prevents accidental cross-app access)
- Potential billing impact (if Task Management is paid add-on) ^[5] ^[9]

Flow

Step 1: Ola Initiates Cross-App Request

- Ola goes to "Team" → Peter's profile
- Clicks "Add to another app"
- Modal: "Select application"
 - Options: Task Management, Gym Management, Bookings (apps activated for this org)
 - Grayed out: Apps not activated for org

Step 2: Select App + Role

- Ola selects "Task Management"
- Role dropdown: "Contributor", "Project Manager", "Viewer" ^[7]

- Message: "Peter, you're invited to collaborate on Task Management"
- Clicks "Request Access"

Step 3: Backend Check - Requires Org Admin

Backend validates:

1. ✓ Ola has `inventory.manage_members` but NOT `workspace.manage_products`
2. ✗ Cross-app assignment requires org owner/admin approval
3. Returns: `{ requiresApproval: true, approvers: ["usr_100", "usr_101"] }`

Step 4: Admin Approval Flow

- System sends notification to org owners/admins:
 - "Ola requested to add Peter to Task Management as Contributor"
 - Approvers click "Review" → see request details
 - Buttons: "Approve" / "Deny"
- If approved:
 - Create `orgApplicationMembers` row for (org, Task Management, `userId=Peter`)
 - Notify Ola + Peter: "✓ Peter added to Task Management"
- If denied:
 - Notify Ola: "Your request was denied by Admin"

Step 5: Peter's Access

- Peter now has two app assignments:
 - Inventory (Branch A, Cashier)
 - Task Management (Contributor, no branch concept)
- Peter switches apps via app launcher: `app.orvionhub.localhost`^[9] ^[5]

3. Database Schema (Convex)

3.1 Core Tables

```
// users - personal accounts (existing)
users: defineTable({
  email: v.string(),
  emailNormalized: v.string(),
  name: v.string(),
  avatarUrl: v.optional(v.string()),
  emailVerified: v.boolean(),
  createdAt: v.number(),
  updatedAt: v.number(),
})
.index("by_email", ["emailNormalized"])
.index("by_email_verified", ["emailNormalized", "emailVerified"])
```

```

// orgMembers - Organization-level membership (Layer 1)
orgMembers: defineTable({
  organizationId: v.id("organizations"),
  userId: v.id("users"),
  role: v.string(), // "owner" | "admin" | "member"
  status: v.string(), // "invited" | "active" | "suspended" | "removed"
  invitedBy: v.optional(v.id("users")),
  invitedAt: v.optional(v.number()),
  acceptedAt: v.optional(v.number()),
  suspendedAt: v.optional(v.number()),
  removedAt: v.optional(v.number()),
  createdAt: v.number(),
  updatedAt: v.number(),
})
  .index("by_organization", ["organizationId"])
  .index("by_user", ["userId"])
  .index("by_organization_user", ["organizationId", "userId"])
  .index("by_user_status", ["userId", "status"])
  .index("by_organization_status", ["organizationId", "status"])

// orgApplicationMembers - App-branch assignments (Layer 2)
orgApplicationMembers: defineTable({
  organizationId: v.id("organizations"),
  applicationId: v.id("applications"),
  branchId: v.optional(v.id("branches")), // nullable for app-level roles (e.g., Task)
  userId: v.id("users"),
  role: v.string(), // "cashier" | "manager" | "contributor" | "viewer" etc.
  permissions: v.array(v.string()), // explicit permissions: ["inventory.sell", "inventory.buy"]
  status: v.string(), // "invited" | "active" | "suspended" | "removed"
  assignedBy: v.id("users"),
  assignedAt: v.number(),
  suspendedAt: v.optional(v.number()),
  removedAt: v.optional(v.number()),
  transferredFrom: v.optional(v.id("branches")), // audit: previous branchId
  createdAt: v.number(),
  updatedAt: v.number(),
})
  .index("by_organization", ["organizationId"])
  .index("by_user", ["userId"])
  .index("by_organization_application", ["organizationId", "applicationId"])
  .index("by_organization_application_branch", ["organizationId", "applicationId", "branchId"])
  .index("by_user_application", ["userId", "applicationId"])
  .index("by_user_status", ["userId", "status"])

// workspaceInvitations - Unified invitations (org + app-branch)
workspaceInvitations: defineTable({
  organizationId: v.id("organizations"),
  applicationId: v.optional(v.id("applications")), // null for org-only invites
  branchId: v.optional(v.id("branches")), // null for app-level invites
  email: v.string(),
  emailNormalized: v.string(),
  inviteeUserId: v.optional(v.id("users")), // resolved after acceptance
  role: v.string(),
  permissions: v.array(v.string()),
  tokenHash: v.string(),
})

```

```

status: v.string(), // "pending" | "accepted" | "declined" | "expired" | "revoked"
invitedBy: v.id("users"),
expiresAt: v.number(),
acceptedAt: v.optional(v.number()),
declinedAt: v.optional(v.number()),
revokedAt: v.optional(v.number()),
createdAt: v.number(),
}))
.index("by_token_hash", ["tokenHash"])
.index("by_organization", ["organizationId"])
.index("by_email_status", ["emailNormalized", "status"])
.index("by_invitee_user", ["inviteeUserId"])
.index("by_organization_application_branch", ["organizationId", "applicationId", "branchId"])

// branches - Physical locations (existing, enhanced)
branches: defineTable({
  organizationId: v.id("organizations"),
  applicationId: v.id("applications"),
  name: v.string(),
  code: v.optional(v.string()),
  address: v.optional(v.string()),
  phone: v.optional(v.string()),
  status: v.string(), // "active" | "archived"
  createdAt: v.number(),
  updatedAt: v.number(),
}))
.index("by_organization", ["organizationId"])
.index("by_organization_application", ["organizationId", "applicationId"])
.index("by_organization_status", ["organizationId", "status"])

// notifications - In-app notifications (existing, enhanced)
notifications: defineTable({
  userId: v.id("users"),
  type: v.string(), // "invitation.received" | "branch.transferred" | "role.changed"
  title: v.string(),
  body: v.string(),
  link: v.optional(v.string()),
  metadata: v.optional(v.any()), // { invitationId, branchId, fromRole, toRole }
  read: v.boolean(),
  readAt: v.optional(v.number()),
  createdAt: v.number(),
}))
.index("by_user", ["userId"])
.index("by_user_read", ["userId", "read"])
.index("by_user_created", ["userId", "createdAt"])

// auditLogs - Comprehensive audit trail (existing, enhanced)
auditLogs: defineTable({
  organizationId: v.optional(v.id("organizations")),
  applicationId: v.optional(v.id("applications")),
  branchId: v.optional(v.id("branches")),
  actorUserId: v.optional(v.id("users")),
  eventType: v.string(), // "workspace.member_invited" | "branch.member_added" | "branch.member_removed"
  entityType: v.string(), // "member" | "invitation" | "branch"
  entityId: v.optional(v.string()), // membershipId, invitationId, etc.
  severity: v.string(), // "low" | "medium" | "high" | "critical"
});

```

```

    metadata: v.optional(v.any()), // { fromBranchId, toBranchId, fromRole, toRole }
    requestId: v.optional(v.string()),
    ipAddress: v.optional(v.string()),
    userAgent: v.optional(v.string()),
    createdAt: v.number(),
  })
  .index("by_organization", ["organizationId"])
  .index("by_organization_created", ["organizationId", "createdAt"])
  .index("by_actor", ["actorUserId"])
  .index("by_entity", ["entityType", "entityId"])

```

3.2 Index Strategy

Critical composite indexes for performance:

Query Pattern	Index
Find user's org memberships	orgMembers.by_user
Find all members in org	orgMembers.by_organization
Check if user is member of org	orgMembers.by_organization_user
Find user's app-branch assignments	orgApplicationMembers.by_user_application
Find all members in branch	orgApplicationMembers.by_organization_application_branch
Lookup invitation by token	workspaceInvitations.by_token_hash
Find pending invites for email	workspaceInvitations.by_email_status
Audit logs by org	auditLogs.by_organization

4. API Endpoints (Fastify)

4.1 Invitation Endpoints

```

// POST /v1/workspaces/:workspaceId/invitations
// Create new invitation (org-level or app-branch-level)
{
  email: string,
  role: string,
  applicationId?: string,
  branchId?: string,
  permissions?: string[],
  expiresInDays?: number,
  message?: string
}
→ { invitationId: string, token: string }

// GET /v1/invitations/:token
// Get invitation details (public, no auth required for token validation)
→ {

```

```

workspace: { id, name, logoUrl },
inviter: { name, avatarUrl },
application?: { name, key },
branch?: { name },
role: string,
permissions: string[],
expiresAt: number,
status: "pending" | "accepted" | ...
}

// POST /v1/invitations/:token/accept
// Accept invitation (authenticated user)
→ { success: true, redirectUrl: string }

// POST /v1/invitations/:token/accept-with-signup
// Accept + create account (unauthenticated user)
{
  name: string,
  password: string,
  agreeToTerms: boolean
}
→ { success: true, redirectUrl: string }

// POST /v1/invitations/:token/decline
// Decline invitation
→ { success: true }

// GET /v1/workspaces/:workspaceId/invitations
// List all invitations for workspace (requires manage_members permission)
→ { invitations: Invitation[] }

// POST /v1/workspaces/:workspaceId/invitations/:invitationId/resend
// Resend invitation email
→ { success: true }

// POST /v1/workspaces/:workspaceId/invitations/:invitationId/revoke
// Revoke pending invitation
→ { success: true }

```

4.2 Membership Management Endpoints

```

// GET /v1/workspaces/:workspaceId/products/:applicationId/branches/:branchId/members
// List all active members in a branch
→ { members: Member[] }

// GET /v1/workspaces/:workspaceId/members/:membershipId
// Get specific org member details
→ { member: OrgMember }

// PATCH /v1/workspaces/:workspaceId/members/:membershipId
// Update org member role (owner/admin/member)
{
  role: "owner" | "admin" | "member",
  reason?: string
}

```

```

}
→ { success: true }

// POST /v1/workspaces/:workspaceId/products/inventory/members/:membershipId/transfer
// Transfer member between branches (same app)
{
  toBranchId: string,
  newRole?: string,
  effectiveDate?: string,
  message?: string,
  notifyImmediately: boolean
}
→ { success: true, transferredAt: number }

// POST /v1/workspaces/:workspaceId/products/inventory/members/:membershipId/suspend
// Suspend member (temporary access revocation)
{
  reason?: string,
  durationDays?: number
}
→ { success: true }

// POST /v1/workspaces/:workspaceId/members/:membershipId/remove
// Remove member from org (revokes all app-branch access)
{
  reason?: string,
  transferOwnership?: boolean // if removing owner
}
→ { success: true }

```

4.3 User Search Endpoint

```

// GET /v1/users/search?email=:email
// Search for user by email (for invite autocomplete)
// Auth required: any authenticated user
→ {
  hasAccount: boolean,
  user?: {
    userId: string,
    name: string,
    avatarUrl?: string,
    publicOrgs?: { organizationId, name, role }[] // only public orgs
  }
}

```

5. Permission Model

5.1 Role Hierarchy

Organization-level roles (orgMembers table): ^[14] ^[7]

Role	Permissions
Owner	Full org control: manage members, billing, settings, delete org, transfer ownership
Admin	Manage members, activate apps, manage branches, view billing
Member	View org profile, accept app assignments, view own activity

Application-level roles (orgApplicationMembers table): ^[7]

App	Roles	Key Permissions
Inventory	Owner, Manager, Cashier, Stock Keeper, Viewer	inventory.sell, inventory.manage_products, inventory.view_reports, inventory.receive_stock
Task Management	Owner, Project Manager, Contributor, Viewer	tasks.create, tasks.assign, tasks.view_all
Gym Management	Owner, Trainer, Receptionist, Viewer	gym.manage_members, gym.book_sessions, gym.view_schedule

Branch-scoped permissions: ^[8]

- Permissions can be scoped to specific branches:
 - inventory.sell:branch_br_789 (can sell only at Branch A)
 - inventory.view_stock:branch_br_790 (can view stock only at Branch B)
- Or global across all branches in app:
 - inventory.sell:* (can sell at any branch)

5.2 Permission Evaluation Flow

For every API request: ^[14] ^[8]

1. Authenticate user (session cookie → userId)
2. Resolve workspace context (from URL or session)
3. Check org membership:
 - Query orgMembers.by_organization_user (orgId, userId)
 - Ensure status = "active"
4. Check app entitlement:
 - Query orgApplications.by_organization_application (orgId, appId)
 - Ensure status = "active"

5. Check app-branch membership:

- Query `orgApplicationMembers.by_organization_application_branch (orgId, appId, branchId, userId)`
- Ensure `status = "active"`

6. Evaluate permission:

- Check if required permission is in member's `permissions` array
- Or check if role grants implicit permission (e.g., "Manager" → all inventory permissions)

7. Authorize or deny:

- If pass: execute query
- If fail: return 403 Forbidden

6. Notification System

6.1 Notification Types

Event	Type	Recipients	Channels
Invitation sent	<code>invitation.sent</code>	Invitee	Email + in-app (if has account)
Invitation accepted	<code>invitation.accepted</code>	Inviter, org admins	In-app + email
Invitation declined	<code>invitation.declined</code>	Inviter	In-app
Invitation expired	<code>invitation.expired</code>	Inviter	In-app
Member transferred	<code>branch.member_transferred</code>	Transferred user, inviter	In-app + email
Role changed	<code>branch.role_changed</code>	Affected user, inviter	In-app + email
Member removed	<code>branch.member_removed</code>	Removed user, org admins	In-app + email
Member suspended	<code>branch.member_suspended</code>	Suspended user, org admins	In-app + email

6.2 Email Templates

Template: Invitation Sent

```
Subject: {{inviterName}} invited you to join {{workspaceName}} as {{role}} ({{applicationName}})

Hi {{inviteeName}},

{{inviterName}} ({{inviterRole}}) has invited you to join {{workspaceName}} as a {{role}} ({{applicationName}}).

Role: {{role}}
```

```
Permissions: {{permissionList}}
Workspace: {{workspaceName}}
Application: {{applicationName}}
Branch: {{branchName}}
```

```
Message from {{inviterName}}:
"{{customMessage}}"
```

This invitation expires on {{expiresAt}}.

[Review Invitation] → {{invitationLink}}

If you don't have an Orviohub account, you'll create one when you accept.

```
---
Orviohub
Abuja, FCT, NG
Unsubscribe | Privacy Policy
```

Template: Member Transferred

```
Subject: You've been transferred to {{newBranchName}} - {{workspaceName}}
```

```
Hi {{memberName}},
```

```
{{inviterName}} has transferred you from {{oldBranchName}} to {{newBranchName}} as {{roleName}}
```

```
Effective date: {{effectiveDate}}
```

```
New role: {{newRole}}
New branch: {{newBranchName}}
Application: {{applicationName}}
```

```
Message from {{inviterName}}:
"{{customMessage}}"
```

```
[View {{newBranchName}} Dashboard] → {{branchDashboardLink}}
```

```
---
Orviohub
Abuja, FCT, NG
```

6.3 In-App Notification Center

UI Components: ^[15]

- Bell icon in header (shows unread count badge)
- Dropdown panel:
 - Tabs: "All" | "Unread" | "Invitations" | "Transfers"
 - List items: Icon + title + body + timestamp + "Mark as read"

- Empty state: "No notifications"
- Mark all as read button
- Infinite scroll (load more)

Real-time updates:

- Convex subscription: `useQuery(notifications.by_user, { userId })`
- Or WebSocket push for instant delivery

7. Audit Logging

7.1 Audit Event Catalog

Event Type	Severity	Metadata Fields
workspace.member_invited	medium	invitationId, email, role, branchId
workspace.invitation_accepted	medium	invitationId, userId, role
workspace.invitation_declined	low	invitationId, email
workspace.invitation_revoked	medium	invitationId, revokedBy
branch.member_added	medium	membershipId, userId, branchId, role
branch.member_transferred	medium	membershipId, userId, fromBranchId, toBranchId, fromRole, toRole
branch.role_changed	medium	membershipId, userId, fromRole, toRole
branch.member_suspended	high	membershipId, userId, reason, durationDays
branch.member_removed	high	membershipId, userId, removedBy, reason
workspace.member_removed	critical	membershipId, userId, removedBy, reason, wasOwner
workspace.ownership_transferred	critical	fromUserId, toUserId, newOwnerRole

7.2 Audit Log Query API

```
// GET /v1/workspaces/:workspaceId/audit-logs
// Query audit logs (requires admin/owner role)
{
  entityType?: "member" | "invitation" | "branch",
  entityId?: string,
  eventType?: string,
  actorUserId?: string,
  startDate?: number,
  endDate?: number,
  severity?: "low" | "medium" | "high" | "critical",
  limit?: number,
  cursor?: string
}
```

```
}  
→ {  
  logs: AuditLog[],  
  nextCursor?: string  
}
```

UI: Audit log viewer in workspace settings → "Security activity"^[15]

8. Security & Validation Rules

8.1 Invitation Security

- **Token hashing:** Store only SHA-256 hash of token in DB (never store plain token)^[3] ^[4]
- **Single-use:** Token invalidated after acceptance/decline^[3]
- **Expiry:** Default 7 days, configurable (max 30 days)^[4]
- **Email binding:** Only the invited email address can accept (check authenticated user's email matches)^[3]
- **Rate limiting:** Max 100 pending invitations per org (prevent spam)^[3]
- **Duplicate prevention:** Reject if active pending invitation exists for same email + branch

8.2 Membership Constraints

- **Owner protection:** Cannot remove last owner from org^[16] ^[^17]
- **Ownership transfer:** Requires re-authentication + explicit confirmation^[^17] ^[16]
- **Cross-app isolation:** User in Inventory app cannot be assigned to Task Management without explicit cross-app invitation (requires org admin approval)^[5]
- **Branch scope:** User can only be assigned to branches within the same org + app^[1]
- **Free Trial enforcement:** Free Trial orgs limited to 1 branch per app (transfers don't change count, but new branch creation blocked)^[9]

8.3 IDOR Prevention (Insecure Direct Object Reference)

- **Never trust client-supplied IDs:** Always validate membership before accessing resources^[^18] ^[14]
- **Workspace binding:** Every query must include `organizationId` filter
- **Composite indexes:** Use `by_organization_application_branch` indexes to enforce scoping
- **Request context middleware:** Attach `workspaceId`, `userId`, `permissions` to request object after auth

Example middleware:

```
// hooks/require-membership.ts
export async function requireMembership(fastify: FastifyInstance) {
  fastify.addHook('preHandler', async (request, reply) => {
    const { workspaceId } = request.params;
    const { userId } = request.user; // from session

    const membership = await convex.query(
      orgMembers.getByOrganizationUser,
      { organizationId: workspaceId, userId }
    );

    if (!membership || membership.status !== 'active') {
      return reply.code(403).send({ error: 'Not a member of this workspace' });
    }

    request.workspaceMembership = membership;
  });
}
```

9. UI/UX Specifications

9.1 Team Management Screen

URL: `inventory.orv.iohub.localhost/settings/team` ^[10] ^[15]

Layout:

- Header: "Team - Branch A" + "Invite Member" button
- Tabs: "All Members" | "Pending Invitations" | "Transferred"
- Search bar: Filter by name/email/role
- Table columns:
 - Name (avatar + name + email)
 - Role (dropdown if editable)
 - Branch (if multi-branch)
 - Status (Active / Invited / Suspended)
 - Last Active (relative time)
 - Joined Date
 - Actions (: menu: Transfer, Change Role, Suspend, Remove)

Empty states:

- No members: "No members yet. Invite your first team member!" + "Invite Member" button
- No pending invites: "No pending invitations"

9.2 Invite Modal

Components:^[19]^[20]

1. Email input:

- Typeahead autocomplete (from previous invites)
- Validation: valid email format
- Real-time user search (debounced 300ms)

2. User preview (if found):

- Avatar + name + "Orviahub member" badge
- Current orgs (if public): "Also works at: Org A, Org B"

3. Role selector:

- Dropdown with role descriptions (e.g., "Cashier: Can sell products, view stock")

4. Branch selector:

- Dropdown (defaults to current branch)
- Shows branch name + address (if available)

5. Message textarea (optional)

6. Expiry selector:

- Dropdown: 1 day, 3 days, 7 days (default), 14 days, 30 days

7. Review section (before send):

- Summary: "Invite john.doe@gmail.com as Cashier in Branch A for 7 days"
- Permissions list: "Can: sell products, view stock, process orders"

8. Buttons:

- "Send Invitation" (primary)
- "Cancel" (secondary)

9.3 Invitation Acceptance Screen

URL: `accounts.orviahub.localhost/invitations/:token[^20]`^[4]

Layout:

- Centered card (max-width 500px)
- Workspace logo (top)
- Title: "You're invited to join {{workspaceName}}"
- Inviter info: "{{inviterName}} invited you"
- Role + app + branch details (with icons)
- Permissions list (bullet points)

- Custom message (in quote block)
- Expiry countdown: "Expires in {{days}} days"
- **If has account:**
 - "Sign in to accept" button (redirects to login → back here)
 - "Decline" link (below button)
- **If no account:**
 - Signup form (name, email read-only, password, terms checkbox)
 - "Create account & accept" button

Post-acceptance:

- Success message: "✓ You've joined {{workspaceName}} as {{role}}!"
- Auto-redirect to dashboard after 3s, or "Go to Dashboard" button

9.4 Transfer Modal

Components:

1. From/To branches:

- From: read-only (current branch name)
- To: dropdown (all branches in org + app)

2. Role selector:

- Dropdown (defaults to current role)
- Shows permission changes (diff view: "+ manage_stock, - sell")

3. Effective date:

- Date picker (min: today, max: 90 days in future)

4. Message to member (optional)

5. Notification toggle:

- Checkbox: "Notify member immediately" (default: checked)

6. Review:

- Summary: "Transfer Peter Johnson from Branch A to Branch B as Manager"
- Effective date: "Starts: Sep 10, 2026"

7. Buttons:

- "Confirm Transfer" (primary, destructive style)
- "Cancel" (secondary)

9.5 Notification Center

UI Components:^[15]

- Bell icon in header (top-right)
- Badge: Red circle with unread count (max "9+")
- Dropdown panel (320px width):
 - Header: "Notifications" + "Mark all as read" link
 - Tabs: "All" | "Unread" | "Invitations"
 - List:
 - Icon (based on type: invitation, transfer, role change)
 - Title (bold): "New invitation from Code X Stores"
 - Body (truncated): "Ola invited you as Cashier in Inventory (Branch A)"
 - Timestamp: "2h ago"
 - Unread indicator (blue dot)
 - Footer: "View all notifications" link → `/notifications`
- Empty state: "No notifications"

Full notifications page:

- URL: `accounts.orvionhub.localhost/notifications`
- Infinite scroll list
- Filter by type
- Bulk actions: "Mark all as read", "Clear all"

10. Testing Plan

10.1 Unit Tests

```
// Invitation creation
test('creates invitation with valid email and role', async () => {
  const invitation = await createInvitation({
    organizationId: 'ws_123',
    email: 'john@example.com',
    role: 'cashier',
    branchId: 'br_789',
    invitedBy: 'usr_100',
  });
  expect(invitation.status).toBe('pending');
  expect(invitation.expiresAt).toBeGreaterThan(Date.now());
});

test('rejects duplicate pending invitation', async () => {
  await createInvitation({ email: 'john@example.com', ... });
});
```

```

    await expect(createInvitation({ email: 'john@example.com', ... }))
      .rejects.toThrow('Pending invitation already exists');
  });

test('hashes invitation token before storing', async () => {
  const invitation = await createInvitation({ ... });
  const stored = await db.workspaceInvitations.get(invitation.id);
  expect(stored.tokenHash).toMatch(/^[a-f0-9]{64}$/); // SHA-256 hex
  expect(stored.token).toBeUndefined(); // never store plain token
});

// Membership transfer
test('transfers member between branches', async () => {
  const member = await createMember({ branchId: 'br_789', role: 'cashier' });
  await transferMember(member.id, { toBranchId: 'br_790', newRole: 'manager' });
  const updated = await getMember(member.id);
  expect(updated.branchId).toBe('br_790');
  expect(updated.role).toBe('manager');
  expect(updated.transferredFrom).toBe('br_789');
});

test('prevents cross-app transfer without approval', async () => {
  const member = await createMember({ applicationId: 'app_inventory' });
  await expect(
    transferMember(member.id, { applicationId: 'app_task_management' })
  ).rejects.toThrow('Cross-app transfer requires admin approval');
});

// Permission evaluation
test('grants access to branch-scoped permission', async () => {
  const member = await createMember({
    branchId: 'br_789',
    permissions: ['inventory.sell:branch_br_789'],
  });
  const hasAccess = await checkPermission(member.userId, 'inventory.sell', { branchId: 'br_789' });
  expect(hasAccess).toBe(true);
});

test('denies access to different branch', async () => {
  const member = await createMember({
    branchId: 'br_789',
    permissions: ['inventory.sell:branch_br_789'],
  });
  const hasAccess = await checkPermission(member.userId, 'inventory.sell', { branchId: 'br_790' });
  expect(hasAccess).toBe(false);
});

```

10.2 Integration Tests

```

// Full invitation + acceptance flow
test('end-to-end invitation and acceptance', async () => {
  // Owner creates invitation
  const invitation = await api.post('/v1/workspaces/ws_123/invitations', {
    email: 'john@example.com',
  });

```

```

    role: 'cashier',
    branchId: 'br_789',
  });
  expect(invitation.status).toBe(200);

  // Invitee accepts (no account)
  const signupResponse = await api.post(`/v1/invitations/${invitation.token}/accept`, {
    name: 'John Doe',
    password: 'SecurePass123!',
    agreeToTerms: true,
  });
  expect(signupResponse.status).toBe(200);

  // Verify membership created
  const member = await db.orgApplicationMembers.getByOrganizationApplicationBranch({
    organizationId: 'ws_123',
    applicationId: 'app_inventory',
    branchId: 'br_789',
    userId: signupResponse.userId,
  });
  expect(member).toBeDefined();
  expect(member.role).toBe('cashier');

  // Verify notification sent
  const notifications = await db.notifications.by_user({ userId: signupResponse.userId });
  expect(notifications.some(n => n.type === 'invitation.received')).toBe(true);

  // Verify audit log
  const auditLogs = await db.auditLogs.by_organization({ organizationId: 'ws_123' });
  expect(auditLogs.some(l => l.eventType === 'workspace.member_invited')).toBe(true);
  expect(auditLogs.some(l => l.eventType === 'workspace.invitation_accepted')).toBe(true);
});

// Transfer flow
test('transfer member between branches', async () => {
  const member = await createMember({ branchId: 'br_789', role: 'cashier' });

  const transferResponse = await api.post(
    `/v1/workspaces/ws_123/products/inventory/members/${member.id}/transfer`,
    { toBranchId: 'br_790', newRole: 'manager' }
  );
  expect(transferResponse.status).toBe(200);

  const updated = await getMember(member.id);
  expect(updated.branchId).toBe('br_790');
  expect(updated.role).toBe('manager');

  // Verify notifications
  const memberNotifications = await db.notifications.by_user({ userId: member.userId });
  expect(memberNotifications.some(n => n.type === 'branch.member_transferred')).toBe(true);
});

```

10.3 Security Tests

```
// IDOR prevention
test('prevents accessing another workspace member', async () => {
  const attacker = await createUser();
  const victimMemberId = 'mem_victim'; // belongs to different org

  const response = await api
    .withUser(attacker)
    .get(`/v1/workspaces/ws_other/members/${victimMemberId}`);
  expect(response.status).toBe(403);
  expect(response.body.error).toBe('Not a member of this workspace');
});

// Token security
test('rejects expired invitation token', async () => {
  const invitation = await createInvitation({ expiresAt: Date.now() - 1000 }); // expired
  const response = await api.post(`/v1/invitations/${invitation.token}/accept`);
  expect(response.status).toBe(400);
  expect(response.body.error).toBe('Invitation expired');
});

test('rejects revoked invitation token', async () => {
  const invitation = await createInvitation({});
  await revokeInvitation(invitation.id);
  const response = await api.post(`/v1/invitations/${invitation.token}/accept`);
  expect(response.status).toBe(400);
  expect(response.body.error).toBe('Invitation revoked');
});

// Owner protection
test('prevents removing last owner', async () => {
  const lastOwner = await createOrgMember({ role: 'owner', organizationId: 'ws_123' });
  const response = await api.post(`/v1/workspaces/ws_123/members/${lastOwner.id}/remove`);
  expect(response.status).toBe(400);
  expect(response.body.error).toBe('Cannot remove the last owner');
});
```

10.4 Performance Tests

```
// Load test: 1000 concurrent invitations
test('handles 1000 concurrent invitation creations', async () => {
  const promises = Array.from({ length: 1000 }).map(() => {
    api.post('/v1/workspaces/ws_123/invitations', {
      email: `user${Math.random()}@example.com`,
      role: 'cashier',
      branchId: 'br_789',
    })
  });
  const results = await Promise.allSettled(promises);
  const successCount = results.filter(r => r.status === 'fulfilled' && r.value.status === 201).length;
  expect(successCount).toBeGreaterThan(950); // allow some rate-limit failures
});
```

```
// Index performance
test('queries org members in <50ms', async () => {
  const startTime = Date.now();
  const members = await db.orgMembers.by_organization({ organizationId: 'ws_123' });
  const duration = Date.now() - startTime;
  expect(duration).toBeLessThan(50);
  expect(members.length).toBeGreaterThan(0);
});
```

11. Implementation Phases

Phase 1: Data Model & Core APIs (Week 1)

Deliverables:

- ✓ Convex schema: orgMembers, orgApplicationMembers, workspaceInvitations, enhanced branches, notifications, auditLogs
- ✓ Indexes for all query patterns
- ✓ API endpoints:
 - POST /v1/workspaces/:workspaceId/invitations
 - GET /v1/invitations/:token
 - POST /v1/invitations/:token/accept
 - POST /v1/invitations/:token/accept-with-signup
 - GET /v1/users/search
- ✓ Unit tests for invitation creation + acceptance

Success criteria:

- Can create invitation via API
- Can accept invitation (with/without existing account)
- Audit logs + notifications generated

Phase 2: Team Management UI (Week 2)

Deliverables:

- ✓ Team management screen (/settings/team)^[10] ^[15]
- ✓ Invite modal (email search, role/branch selection, review)^[19]^[20]
- ✓ Invitation acceptance screen (/invitations/:token)^[^20] ^[4]
- ✓ Pending invitations list + actions (resend, revoke)
- ✓ Email templates (invitation sent, accepted)
- ✓ Integration tests for full flow

Success criteria:

- Branch owner can invite staff via UI
- Invitee can accept (with/without account)
- Notifications sent + audit logged

Phase 3: Transfer & Role Management (Week 3)

Deliverables:

- ✓ Transfer modal (from/to branch, role change, effective date) ^[11]
- ✓ API: POST /v1/workspaces/:workspaceId/products/:appId/members/:id/transfer
- ✓ Role change API: PATCH /v1/workspaces/:workspaceId/members/:id
- ✓ Transfer/role change notifications + emails
- ✓ Audit logs for transfers + role changes
- ✓ Member list actions (suspend, remove, change role) ^[10]
- ✓ Owner protection logic (cannot remove last owner) ^[16] ^[^17]

Success criteria:

- Can transfer staff between branches
- Can change roles
- Cannot remove last owner
- All actions audited + notified

Phase 4: Cross-App & Advanced Features (Week 4)

Deliverables:

- ✓ Cross-app invitation flow (requires admin approval) ^[5]
- ✓ Admin approval UI (pending requests, approve/deny)
- ✓ Branch-scoped permissions (inventory.sell:branch_br_789) ^[8]
- ✓ Permission evaluation middleware
- ✓ Notification center UI (bell icon, dropdown, full page) ^[15]
- ✓ Audit log viewer (filterable, searchable) ^[15]
- ✓ Performance optimizations (index usage, query caching)

Success criteria:

- Can add staff to multiple apps (with approval)
- Permissions correctly scoped to branches
- Notifications + audit logs working end-to-end

Phase 5: Security Hardening & Polish (Week 5)

Deliverables:

- ✓ Security tests (IDOR, token expiry, rate limiting)[¹⁸][¹⁴][³]
- ✓ Load tests (1000 concurrent invites, large orgs)
- ✓ Error handling + user-friendly error messages
- ✓ Accessibility (a11y) audit (keyboard nav, screen readers)
- ✓ Mobile responsiveness (team list, modals)
- ✓ Documentation (API docs, user guides)

Success criteria:

- All security tests pass
- No critical vulnerabilities
- UI works on mobile + desktop
- Docs complete

12. Migration from Current Model

If you already have a simpler team model (e.g., only org-level members), migrate as follows:

Step 1: Backfill orgApplicationMembers

```
-- For each existing org member, create default app-branch assignment
INSERT INTO orgApplicationMembers (organizationId, applicationId, branchId, userId, role, status, assignedBy, assignedAt)
SELECT
  om.organizationId,
  'app_inventory' AS applicationId, -- or derive from existing data
  b.id AS branchId, -- default branch or first branch
  om.userId,
  om.role AS role, -- map org role to app role
  'active' AS status,
  om.invitedBy AS assignedBy,
  om.acceptedAt AS assignedAt
FROM orgMembers om
JOIN branches b ON b.organizationId = om.organizationId
WHERE om.status = 'active'
  AND b.status = 'active'
LIMIT 1; -- one branch per member (adjust logic as needed)
```

Step 2: Update Authorization Middleware

- Add check for `orgApplicationMembers` in addition to `orgMembers`
- Graceful fallback: if no `orgApplicationMembers` row exists, grant legacy access (for migration period)

Step 3: Deploy New UI

- Roll out team management screen to beta users
- Monitor for issues
- Gradual rollout to all users

13. Key Decisions & Rationale

Decision	Rationale
Hybrid model (org + app-branch)	Balances GitHub-style org membership with operational branch-level flexibility ^[1] ^[2] ^[3]
Invitations require org membership first	Matches GitHub's model: must be org member before joining teams/repos ^[2] ^[3]
Cross-app movement requires approval	Prevents accidental cross-app access, aligns with billing (apps may be paid add-ons) ^[5] ^[9]
Token hashing (not plain storage)	Security best practice: if DB compromised, tokens can't be reused ^[3] ^[4]
7-day default expiry	Matches GitHub standard, balances urgency with flexibility ^[4]
Dual notifications (email + in-app)	Maximizes acceptance rates, matches user expectations ^[1] ^[6]
Branch-scoped permissions	Enables fine-grained access control (e.g., cashier only at Branch A) ^[8]
Audit logs for all membership events	Compliance + debugging, matches enterprise SaaS standards

14. Risks & Mitigations

Risk	Impact	Mitigation
Complexity of hybrid model	Medium	Start with org-only invites, add app-branch layer in Phase 2
Token security vulnerabilities	High	Use established crypto libraries (e.g., <code>crypto</code> in Node.js), hash tokens, never log plain tokens
Performance degradation at scale	Medium	Use composite indexes, paginate large lists, cache frequently accessed data
User confusion (org vs app-branch roles)	Low	Clear UI labels, tooltips, onboarding guide for branch owners
Cross-app access bugs	High	Write extensive tests, require admin approval for cross-app moves
Email deliverability issues	Medium	Use reputable ESP (SendGrid/Resend), monitor bounce rates, provide resend button

15. Success Metrics

Track these KPIs post-launch:

- **Invitation acceptance rate:** Target >70% within 7 days
- **Time to accept:** Median <24 hours
- **Transfer success rate:** >95% without errors
- **Support tickets:** <5 tickets/week related to invitations/transfers
- **Audit log coverage:** 100% of membership events logged
- **Notification open rate:** Email >40%, in-app >80%

Appendix A: Example API Responses

GET /v1/invitations/:token

```
{
  "workspace": {
    "id": "ws_123",
    "name": "Code X Stores",
    "logoUrl": "https://cdn.orviohub.com/logos/ws_123.png"
  },
  "inviter": {
    "name": "Ola",
    "avatarUrl": "https://cdn.orviohub.com/avatars/usr_100.png",
    "role": "owner"
  },
  "application": {
    "id": "app_inventory",
    "name": "Inventory",
    "key": "inventory"
  },
  "branch": {
    "id": "br_789",
    "name": "Branch A",
    "address": "123 Main St, Abuja"
  },
  "role": "cashier",
  "permissions": ["inventory.sell", "inventory.view_stock", "inventory.process_orders"],
  "expiresAt": 1694908800000,
  "status": "pending",
  "message": "Welcome to the team! Your shift starts Monday."
}
```

POST /v1/invitations/:token/accept (response)

```
{
  "success": true,
  "redirectUrl": "/dashboard",
  "workspaceContext": {
    "workspace": {
      "id": "ws_123",
      "name": "Code X Stores"
    },
    "membership": {
      "role": "member",
      "status": "active"
    },
    "products": [
      {
        "key": "inventory",
        "status": "active"
      }
    ],
    "branchAssignments": [
      {
        "branchId": "br_789",
        "branchName": "Branch A",
        "role": "cashier"
      }
    ],
    "permissions": ["inventory.sell", "inventory.view_stock"]
  }
}
```

Appendix B: Email Template HTML (Simplified)

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .button {
      background-color: #4F46E5;
      color: white;
      padding: 12px 24px;
      text-decoration: none;
      border-radius: 6px;
      display: inline-block;
    }
    .footer {
      color: #6B7280;
      font-size: 12px;
      margin-top: 24px;
    }
  </style>
</head>
<body>
```

```

<table width="100%" cellpadding="0" cellspacing="0">
  <tr>
    <td align="center">
      
    </td>
  </tr>
  <tr>
    <td align="center" style="padding: 24px;">
      <h1>You're invited to join {{workspaceName}}</h1>
      <p><strong>{{inviterName}}</strong> ({{inviterRole}}) has invited you as <strong>Application:</strong> {{applicationName}}<br/>
      <strong>Branch:</strong> {{branchName}}</p>
      <blockquote>{{customMessage}}</blockquote>
      <p style="color: #6B7280; font-size: 14px;">This invitation expires on {{expirationDate}}</p>
      <a href="{{invitationLink}}" class="button">Review Invitation</a>
    </td>
  </tr>
  <tr>
    <td class="footer" align="center">
      <p>Orviahub<br/>Abuja, FCT, NG</p>
      <p><a href="{{unsubscribeLink}}">Unsubscribe</a> | <a href="{{privacyLink}}">Privacy Policy</a></p>
    </td>
  </tr>
</table>
</body>
</html>

```

This implementation plan provides a complete, production-ready specification for Orviahub's team and branch management system. It balances GitHub-style UX with your multi-app, multi-branch architecture, ensuring security, scalability, and a smooth user experience. ^{[2] [9] [4] [5] [1] [3]}

**

1. docs/Workspace Management Implementation Plan.pdf
2. <https://docs.github.com/en/rest/teams/members>
3. <https://tessl.io/registry/skills/github/better-auth/skills/organization-best-practices>
4. <https://www.stitchflow.com/user-management/github/manual>
5. <https://www.techbloat.com/how-to-give-access-to-github-repository.html>
6. <https://codeables.dev/article/how-do-saas-companies-handle-parent-child-accounts-for-enterprises>
7. <https://www.scribd.com/document/978888905/GitHub-Onboarding-and-Licensing-Guide>
8. https://note.com/fast_owl436/n/n8970392284fe?hl=en
9. <https://sociilabs.com/blog/saas-multi-tenancy-architecture>
10. https://note.com/nasty_neo/n/n3c46cc258f4b?hl=en
11. <https://devactivity.com/posts/apps-tools/streamlining-github-access-members-vs-outside-collaborators-for-enhanced-engineering-metrics/>
12. <https://old.sectionvtrack.com/article/how-to-add-people-to-github-repo-the-definitive-process>
13. <https://tessl.io/registry/skills/github/slurpyb/skills/organization-best-practices--organization-best-practices>

14. <https://github.com/orgs/community/discussions/188479>
15. <https://note.com/abc1nc/n/n1cdd881e4b38?hl=en>
16. <https://docs.github.com/en/rest/orgs/members?apiVersion=2026-03-10>